Cologne Chip

GateMate™ FPGA

Toolchain Installation User Guide



UG1002

User Guide May 2026

Cologne Chip AG
Eintrachtstr. 113
50668 Köln

Tel.: +49 (0) 221 / 91 24-0
Fax: +49 (0) 221 / 91 24-100

https://colognechip.com
info@colognechip.com

Copyright 2019 - 2026 Cologne Chip AG

All Rights Reserved

The information presented can not be considered as assured characteristics. Data can change without notice. Parts of the information presented may be protected by patent or other rights. Cologne Chip products are not designed, intended, or authorized for use in any application intended to support or sustain life, or for any other application in which the failure of the Cologne Chip product could create a situation where personal injury or death may occur.

Contents

About this Document 4

1 Introduction 5

2 Downloading and Installing the Pre-build Software 7

2.1 Downloading the Pre-build Packages . . . 7

2.2 Software Installation . . . 7

3 Quick Start 9

3.1 Design Synthesis . . . 9

3.1.1 Synthesis of VHDL Sources . . . 9

3.1.2 Synthesis of SystemVerilog Sources . . . 9

3.1.3 Synthesis of Verilog Sources . . . 9

3.2 Design Implemention . . . 11

3.3 FPGA Configuration . . . 12

3.3.1 Configuration of the FPGA via JTAG . . . 12

3.3.2 Configuration of the FPGA via SPI . . . 13

3.3.3 Saving the Bitstream to External Flash Memory . . . 13

A Software Building from Source Code 14

A.1 Building the Synthesis Software . . . 14

A.1.1 Building Yosys for Linux . . . 14

A.1.2 Building GHDL for Linux . . . 15

A.2 Building the Programmer Software . . . 16

About this Document

This user guide describes the toolchain installation for the Cologne Chip GateMate™ series and is part of the GateMate™ documentation collection.

For more information please refer to the following documents:

- Technology Brief of GateMate™ FPGA
- **DS1001** – GateMate™ FPGA CCGM1A1 / CCGM1A2 Datasheet
- **DS1002** – GateMate™ FPGA Programmer Board Datasheet
- **DS1003** – GateMate™ FPGA Evaluation Board Datasheet

Cologne Chip provides a comprehensive technical support. Please visit our website for more information or contact our support team.

Revision History

This datasheet is constantly updated. The latest version of the document can be found following the link below:

UG1002 – GateMate™ FPGA Toolchain Installation User Guide

Date	Remarks
May 2026	Adaptation to `nextpnr` support and revision due to various software updates.
March 2025	Clone repository command slightly changed on page 14.
August 2024	Icarus weblink changed to https://bleyer.org/icarus/.
January 2024	Minor changes in Section 5.3 on page 15.
November 2022	Call of Place & Route added for Windows in Section 5.2 on page 14.
September 2022	Comprehensive revision and notes on configuration via SPI in Section 5.3 from page 15.
August 2022	Comprehensive revision.
April 2022	Pre-build binary option of `openFPGALoader` for Windows added.
March 2022	Initial release.

1 Introduction

This user guide describes the toolchain installation for the Cologne Chip GateMate™ series. It covers building latest software from source or downloading pre-build binaries for Linux, Windows and macOS environments:

darwin-arm64	macOS on Apple Silicon
darwin-x64	macOS on Intel
linux-arm64	Linux on ARM (e.g. Raspberry Pi, embedded devices)
linux-x64	Linux on standard x86_64 systems
windows-x64	Windows 10 / 11 (x86_64)

The GateMate FPGA toolchain is based on the open-source OSS CAD Suite, which bundles together essential tools such as:

yosys	Synthesis framework for Verilog / SystemVerilog/ VHDL
nextpnr-himbaechel	Place and route tool for GateMate devices
gmpack	Bitstream generation utility
openFPGALoader	Programming utility for JTAG, SPI and external flash configuration

In addition, OSS CAD Suite also contains essential supplementary tools such as formal tools, simulators and waveform viewers:

slang	Enhanced SystemVerilog frontend
ghdl	Open-source simulator for the VHDL language (linux-x64, darwin-x64 and darwin-arm64 platforms only)
iverilog	Verilog language compiler and simulator
cocotb	Coroutine based cosimulation library for writing VHDL and Verilog testbenches in Python
verilator	Verilog / SystemVerilog simulator with cycle-accurate behavioral model generator
surfer	Extensible waveform viewer
gtkwave	GTK+ based waveform viewer

An exemplary toolchain flow from design entry to configuration is illustrated in Figure 1.

The Yosys Open SYnthesis Suite is used to perform RTL synthesis. It has extensive Verilog support. VHDL sources can be synthesized via GHDL through the `ghdl-yosys-plugin`.

Synthesis generates a gate-level representation of the design entry in form of a JSON netlist of architecture-specific primitives. It can be passed to the place and route tool

for implementation and bitstream generation. Furthermore, simulation of the resulting netlist with back-annotated timing delays can be done using third-party simulators such as Icarus Verilog together with the GTKWave waveform viewer.

Configuration bitstreams are loaded into the FPGA or an external flash memory via `openFPGALoader`.

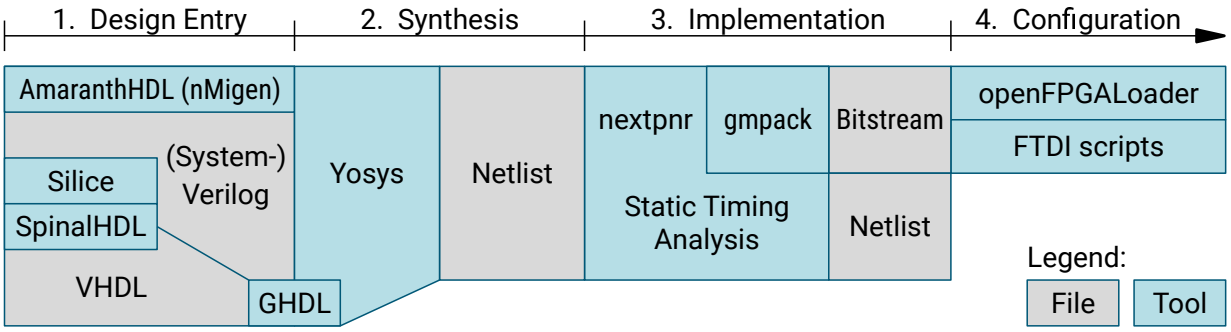


Figure 1: *Exemplary Toolchain Flow*

2 Downloading and Installing the Pre-build Software

2.1 Downloading the Pre-build Packages

The latest pre-built toolchain packages for all major platforms can be found on the official GitHub release page https://github.com/YosysHQ/oss-cad-suite-build/releases.

There are packages for Windows, Linux and maxOS. WSL users are advised to use the Windows packages. Chapter 3 describes the contents of the packages and how to operate the software using the included scripts.

The appropriate archive must be selected based on the architecture and operating system being used:

darwin-arm64	macOS on Apple Silicon
darwin-x64	macOS on Intel
linux-arm64	Linux on ARM (e.g. Raspberry Pi, embedded devices)
linux-x64	Linux on standard x86_64 systems
windows-x64	Windows 10 / 11 (x86_64)

The OSS CAD Suite provides pre-build packages containing all toolchain binaries and sample projects. Please check the availability on the official website.

2.2 Software Installation

The archive can be extracted to any directory on Linux or macOS systems. Next, a shell has to be opened to initialize the environment:

```
$ tar -xvf oss-cad-suite-<arch>-<build>.tgz
$ chmod +x environment
$ source environment
```

After sourcing, the system variable `$PATH` will include all toolchain executables (`yosys`, `nextpnr-himbaechel`, `gmpack`, `openFPGALoader`, etc.).

For example, the following command can be used to verify that everything is set up correctly:

```
$ yosys -V
Yosys 0.58+ (git sha1 ...)
```

If a permanent setup is prefered, the environment initialization line can be added to the shell configuration file (e.g. `.bashrc`, `.zshrc`):

```
$ source /path/to/oss-cad-suite/environment
```

Notes:

Windows users: For the best experience, a Unix-compatible shell (e.g., Git Bash or WSL) is recommended.

Linux users: `libtinfo`, `libffi` and `libz` must be installed.

macOS users: When first launched the application, macOS may prompt allowing unsigned binaries to run. This can be enabled in System Preferences under 'Security & Privacy'.

3 Quick Start

3.1 Design Synthesis

Synthesis converts the HDL source files (VHDL, Verilog or SystemVerilog) into a technology-specific netlist that can be mapped to the FPGA fabric. The GateMate synthesis flow uses GHDL for VHDL sources and Yosys for Verilog / SystemVerilog sources.

3.1.1 Synthesis of VHDL Sources

For VHDL-based projects, GHDL with Yosys integration is used to generate a Verilog netlist:

```
$ ghdl synth -fexplicit -fsynopsys --out=verilog <FILES> -e <top_entity>
```

`-fexplicit`	ensures all signals and generics are explicitly defined.
`-fsynopsys`	enables Synopsys-compatible VHDL parsing.
`<FILES>`	refers to all your VHDL sources (space-separated).
`<top_entity>`	defines the top-level design unit to synthesize.

Once completed, a Verilog output file has been generated that can be passed directly to Yosys for synthesis.

3.1.2 Synthesis of SystemVerilog Sources

For SystemVerilog-bases projects, the slang plugin is used:

```
$ yosys -m slang \
    -ql <logfile> \
    -p '
        read_slang <sources>; # read SystemVerilog files
        [...] # see `synth_gatemate` pass below
    '
```

Further SystemVerilog features are shown with the command ‘`help slang`’.

A full documentation is available at the slang website and at GitHub.

3.1.3 Synthesis of Verilog Sources

The built-in Yosys frontend supports both Verilog and a basic of set SystemVerilog features. The `synth_gatemate` command within Yosys ensures the output is optimized for the GateMate FPGA architecture.

```
$ yosys \
    -ql <logfile> \
    -p '
        read_verilog -defer -sv <sources>; # read verilog files with sv
           ↪  support
        synth_gatemate \
            -top <topmodule>        # top module name
            -luttree                # mandatory: enable luttree support
            -nomx8;                 # mandatory: disable MUX8 support
    write_json <netlist>.json;  # write JSON netlist for implementation
    write_verilog <netlist>.v   # optional: write verilog netlist
'
```

`synth_gatemate` parameters:

`-top <module>`	The specified module is used as top module.
`-vlog <file>`	Writes the design to the specified Verilog file. Writing to an output file is omitted if this parameter is not specified.
`-json <file>`	Writes the design to the specified JSON file. Writing to an output file is omitted if this parameter is not specified.
`-noflatten`	The design is not flattened before synthesis.
`-scopename`	Creates *scopename* attributes when flattening the netlist (required for manual floorplanning in `nextpnr`)
`-nobram`	Block RAM cells `CC_BRAM_20K` and `CC_BRAM_40K` are not used in the output netlist.
`-noaddf`	Full adder cells `CC_ADDF` are not used in the output netlist.
`-nomult`	Multiplier cells `CC_MULT` are not used in the output netlist.
`-nomx8, -nomx4`	Multiplexer cells `CC_MX4` and `CC_MX8` are not used in the output netlist.
`-luttree`	The new LUT tree mapping approach is used.
`-noiopad`	I/O buffer insertion (useful for hierarchical or out-of-context flows) is disabled.
`-noclkbuf`	Automatic clock buffer insertion is disabled.

The resulting `.json` output file is used as input for place and route.

It is strongly recommended to verify the synthesis log output for warnings or errors.

Further, the `stat` command inside Yosys should be called to check resource utilization.

3.2 Design Implemention

The implementation maps the synthesized netlist to the physical GateMate FPGA resources, performing placement, routing and timing analysis. This is handled by `nextpnr-himbaechel`, the backend optimized for GateMate devices.

```
$ nextpnr-himbaechel \
   --device=CCGM1A1        # GateMate device, select CCGM1A1 or CCGM1A2
   --json <filename>.json  # input netlist after synthesis with `yosys`
   -o ccf=<filename>.ccf   # input CCF pin constraints file
   -o out=impl.txt         # output textfile for bitstream generation
   --sdc <filename>.sdc    # optional: input SDC constraints file
   --sdf <filename>.sdf    # optional: write SDF for back-annotated
      ↪ timing simulation
   --router router2        # router, always select `router2`
```

The `.sdc` (Synopsys Design Constraints) output file defines timing requirements such as clocks and paths.

A graphical interface can be launched with the '`--gui`' option to visually inspect the design.

Additional architecture-specific parameters:

`--device <CCGM1A1|CCGM1A2>`	Selection between CCGM1A1 and CCGM1A2.
`--placer <sa|static|heap>`	Selection of the placer algorithm (default: heap).
`--router <router1|router2>`	Selection of the router algorithm (default and recommended: router2)
`-o ccf=<file>`	Name of the constraint file (CCF) defining pin assignments and I/O standards.
`-o allow-unconstrained`	Option to allow unconstrained I/Os remaining unassigned (use with caution).
`-o fpga_mode=<lowpower|economy|speed>`	Selection of the FPGA operation mode.
`-o time_mode=<best|typical|worst>`	Selection of the timing analysis mode.

`-o strategy=<mirror|full|clk1>`	Selection of the multi-die clock placement strategy. `mirror`: mirrored placement for symmetry `full`: full clock network placement `clk1`: single-clock-die routing only
`-o force_die=<ID>`	Force place and route on a specific die (for CCGM1A2 only: 1A or 1B).
`-o no-cpe-cp`	Disable passing CP lines through CPEs.
`-o no-bridges`	Disable using CPEs in bridge mode.

After successful implementation, a placement and routing report is generated. It should be reviewed to ensure timing closure and resource efficiency.

Next, `gmpack` converts the implementation file into a bitstream:

```
$ gmpack \
    --spimode <arg>     # optional: flash SPI mode (single, dual, quad)
    --crcmode <arg>     # optional: CRC error behaviour (check, ignore,
       ↪  unused)
    --reconfig          # optional: enable reconfiguration interface
    --bootaddr <arg>    # optional: boot address for secondary
       ↪ bitstream in flash
    --background        # optional: enables background reconfiguration
       ↪ in flash mode
    <input>.txt         # input textfile after `nextpnr`
    <output>.bit        # output bitfile for programmer
```

3.3 FPGA Configuration

Finally, the generated bitstream file has to be loaded onto the FPGA. This is done with `openFPGALoader` either via JTAG or SPI.

3.3.1 Configuration of the FPGA via JTAG

JTAG configuration is suitable for development and debugging since it provides quick re-programming without requiring flash memory.

```
$ openFPGALoader \
    --index-chain <arg> # optional: device index in JTAG chain
    --freq <arg>        # optional: programmer frequency in Hz (default
       ↪  6M)
    -b <arg>            # gatemate_evb_jtag: jtag, olimex_gatemateebb:
       ↪ dirtyJtag
    <bitfile>           # input bitfile after `gmpack`
```

3.3.2 Configuration of the FPGA via SPI

SPI mode allows configuration directly from a serial interface or flash device.

```
$ openFPGALoader \
    --freq <arg>        # optional: programmer frequency in Hz (default
       ↪  6M)
    -b <arg>            # gatemate_evb_spi: spi
    -m                  # write bitstream in SRAM
    <bitfile>           # input bitfile after `gmpack`
```

3.3.3 Saving the Bitstream to External Flash Memory

To make the design persistent (i.e., automatically loaded at power-up), it can be programmed into the onboard flash memory:

```
$ openFPGALoader \
    --index-chain <arg> # optional: device index in JTAG chain
    --freq <arg>        # optional: programmer frequency in Hz (default
       ↪  6M)
    -b <arg>            # gatemate_evb_jtag: jtag, gatemate_evb_spi:
       ↪ spi, olimex_gatemateebb: dirtyJtag
    -f                  # write bitstream in flash
    <bitfile>           # input bitfile after `gmpack`
```

To verify that the FPGA design is loaded automatically from flash memory, the board can be powered off and on again, or the reset button gets pressed.

A Software Building from Source Code

Advanced users can, as an alternative to the method described in Chapter 2, download the source files from the GitHub server and compile them themselves.

A.1 Building the Synthesis Software

The Yosys Open SYnthesis Suite is used to perform RTL synthesis. VHDL support is only available by installing the `ghdl` and `ghdl-yosys-plugin` extensions.

A.1.1 Building Yosys for Linux

Install prerequisites for Debian/Ubuntu Linux with `apt`:

```
$ sudo apt install build-essential clang bison flex libreadline-dev
   ↪ gawk tcl-dev libffi-dev git graphviz xdot pkg-config python3
   ↪ libboost-system-dev libboost-python-dev libboost-filesystem-dev
   ↪ zlib1g-dev
```

Install prerequisites for Arch-based Linux with `pacman`:

```
$ sudo pacman -S base-devel git tcl zlib
```

Install prerequisites for Red Hat-based Linux, e.g. Fedora, with `dnf`:

```
$ sudo dnf install make clang tcl-devel zlib-devel readline-devel
   ↪ libffi-devel bison flex
```

The following commands download, compile and install the source files regardless of the Linux distribution used. First, clone the repository with `git` and change into the directory:

```
$ git clone https://github.com/YosysHQ/yosys.git --recursive
$ cd yosys
```

Inside the Yosys directory, configure the build system to use a specific compiler such as `gcc` or `clang`.

```
$ make config-clang
```

The typical build and installation process for Linux is invoked with the following two commands:

```
$ make -j$(nproc)
$ sudo make install
```

A.1.2 Building GHDL for Linux

Both `ghdl` and `ghdl-yosys-plugin` are only required for VHDL support in Yosys.

Install prerequisites for Debian/Ubuntu Linux with `apt`:

```
$ sudo apt install gnat zlib1g-dev
```

Install prerequisites for Arch-based Linux with `pacman`:

```
$ sudo pacman -S gcc-ada zlib
```

Install prerequisites for Red Hat-based Linux, e.g. Fedora, with `dnf`:

```
$ sudo dnf install gcc-gnat zlib
```

The following commands download, compile and install the source files regardless of the Linux distribution used. First, clone the repository with `git` and change into the directory:

```
$ git clone https://github.com/ghdl/ghdl.git
$ cd ghdl
```

The configuration, build and installation process is invoked with the following three commands:

```
$ ./configure --prefix=/usr/local
$ make
$ sudo make install
```

Now clone the `ghdl-yosys-plugin` with `git` and change into the directory.

```
$ cd ..
$ git clone https://github.com/ghdl/ghdl-yosys-plugin.git
$ cd ghdl-yosys-plugin
```

The Makefile will find the `ghdl` installation in `/usr/local`. The following command builds the plugin:

```
$ make
```

The output is a shared library (`ghdl.so` on GNU/Linux). To install the module, the library must be copied to `YOSYS_PREFIX/share/yosys/plugins/ghdl.so`, where `YOSYS_PREFIX` is the installation path of Yosys. This can be achieved through a make target:

```
$ sudo make install
```

The library can then be used by Yosys directly, e.g. with:

```
$ yosys -m ghdl
```

Section 3 describes how to synthesize VHDL sources using the GHDL plugin.

A.2 Building the Programmer Software

`openFPGALoader` integrates well into the toolchain. Version v0.7.0 or later is required.

Building `openFPGALoader` for Linux

Install prerequisites for Debian/Ubuntu Linux with `apt`:

```
$ sudo apt install libftdi1-2 libftdi1-dev libhidapi-hidraw0 libhidapi-
   ↪ dev libudev-dev zlib1g-dev cmake pkg-config make g++
```

Install prerequisites for Arch-based Linux with `pacman`:

```
$ sudo pacman -S git cmake make gcc pkgconf libftdi libusb zlib hidapi
```

Install prerequisites for Red Hat-based Linux, e.g. Fedora, with `dnf`:

```
$ sudo dnf install cmake libftdi-devel zlib hidapi-devel systemd-devel
```

The following commands download, compile and install the source files regardless of the Linux distribution used. First, clone the repository with `git` and change into the directory.

```
$ git clone https://github.com/trabucayre/openFPGALoader.git
$ cd openFPGALoader
```

Next, the following three commands prepare `cmake`:

```
$ mkdir build
$ cd build
$ cmake ..
```

The build and installation process is invoked with the following two commands:

```
$ make -j$(nproc)
$ sudo make install
```

GateMate™ FPGA
Toolchain Installation User Guide
UG1002
May 2026

Cologne Chip